Type-Object-Pattern

Carsten Gips (HSBI)

TL;DR
Das Type-Object-Pattern dient dazu, die Anzahl der Klassen auf Code-Ebene zu reduzieren und durch eine Konfiguration zu ersetzen und damit eine höhere Flexibilität zu erreichen.
Dazu werden sogenannte Type-Objects definiert: Sie enthalten genau die Eigenschaften, die in verschiedenen (Unter-) Klassen gemeinsam vorkommen. Damit können diese Eigenschaften aus den ursprünglichen Klassen entfernt und durch eine Referenz auf ein solches Type-Object ersetzt werden. In den Klassen muss man dann nur noch die für die einzelnen Typen individuellen Eigenschaften implementieren. Zusätzlich kann man nun verschiedene (Unter-) Klassen zusammenlegen, da der Typ über das geteilte Type-Object definiert wird (zur Laufzeit) und nicht mehr durch eine separate Klasse auf Code-Ebene repräsentiert werden muss.
Die Type-Objects werden zur Laufzeit mit den entsprechenden Ausprägungen der früheren (Unter-) Klassen angelegt und dann über den Konstruktor in die nutzenden Objekte übergeben. Dadurch teilen sich alle Objekte einer früheren (Unter-) Klasse das selbe Type-Objekt und zeigen nach außen das selbe Verhalten. Die Type-Objects werden häufig über eine entsprechende Konfiguration erzeugt, so dass man beispielsweise unterschiedliche Monsterklassen und -eigenschaften ausprobieren kann, ohne den Code neu kompilieren zu müssen. Man kann sogar eine Art “Vererbung” unter den Type-Objects implementieren.
Videos

- VL Type-Object-Pattern

---

1 Motivation: Monster und spezialisierte Monster

```java
public abstract class Monster {
    protected int attackDamage;
    protected int movementSpeed;

    public Monster(int attackDamage, int movementSpeed) { ... }
    public void attack(Monster m)  { ... }
}

public class Rat extends Monster {
    public Rat() { super(10, 10); }  // Ratten haben 10 Damage und 10 Speed
    @Override public void attack(Monster m)  { ... }
}

public class Gnoll extends Monster { ... }


public static void main(String[] args) {
    Monster harald = new Rat();
    Monster eve = new Gnoll();
    ...
}
```

Sie haben sich eine Monster-Basisklasse geschrieben. Darin gruppieren Sie typische Eigenschaften eines Monsters: Es kann sich mit einer bestimmten Geschwindigkeit bewegen und es kann anderen Monstern bei einem Angriff einen bestimmten Schaden zufügen.

Um nun andere Monstertypen zu erzeugen, greifen Sie zur Vererbung und leiten von der Basisklasse Ihre spezialisierten Monster ab und überschreiben die Defaultwerte und bei Bedarf auch das Verhalten (die Methoden).

Damit entsteht aber recht schnell eine tiefe und verzweigte Vererbungshierarchie, Sie müssen ja für jede Variation eine neue Unterklasse anlegen. Außerdem müssen für jede (noch so kleine) Änderung an den Monster-Eigenschaften viele Klassen editiert und das gesamte Projekt neu kompiliert werden.

Es würde auch nicht wirklich helfen, die Eigenschaften der Unterklassen über deren Konstruktor einstellbar zu machen (die `Rat` könnte in ihrem Konstruktor beispielsweise noch die Werte für Damage und Speed übergeben bekommen). Dann würden die Eigenschaften an allen Stellen im Programm verstreut, wo Sie den Konstruktor aufrufen.

2 Vereinfachen der Vererbungshierarchie (mit Enums als Type-Object)

```
public enum Species { RAT, GNOLL, ... }

public final class Monster {
    private final Species type;
    private int attackDamage;
    private int movementSpeed;

    public Monster(Species type) {
        switch (type) {
            case RAT: attackDamage = 10; movementSpeed = 10; break;
            ...
        }
    }
    public void attack(Monster m)  { ... }
}


public static void main(String[] args) {
    Monster harald = new Monster(Species.RAT);
    Monster eve = new Monster(Species.GNOLL);
    ...
}
```

Die Lösung für die Vermeidung der Vererbungshierarchie: Die Monster-Basisklasse bekommt ein Attribut, welches den Typ des Monsters bestimmt (das sogenannte “Type-Object”). Das könnte wie im Beispiel ein einfaches Enum sein, das in den Methoden des Monsters abgefragt wird. So kann zur Laufzeit bei der Erzeugung der Monster-Objekte durch Übergabe des Enums bestimmt werden, was genau dieses konkrete Monster genau ist bzw. wie es sich verhält.

Im obigen Beispiel wird eine Variante gezeigt, wo das Enum im Konstruktor ausgewertet wird und die Attribute entsprechend gesetzt werden. Man könnte das auch so implementieren, dass man auf die Attribute verzichtet und stattdessen stets das Enum auswertet.

Allerdings ist das Hantieren mit den Enums etwas umständlich: Man muss an allen Stellen, wo das Verhalten der Monster unterschiedlich ist, ein **`switch`**/**`case`** einbauen und den Wert des Type-Objects abfragen. Das

bedeutet einerseits viel duplizierten Code und andererseits muss man bei Erweiterungen des Enums auch *alle* `switch`/`case`-Blöcke anpassen.

3 Monster mit Strategie

```
public final class Species {
    private final int attackDamage;
    private final int movementSpeed;
    private final int xp;

    public Species(int attackDamage, int movementSpeed, int xp) { ... }
    public void attack(Monster m)  { ... }
}

public final class Monster {
    private final Species type;
    private int xp;

    public Monster(Species type) { this.type = type;  xp = type.xp(); }
    public int movementSpeed() { return type.movementSpeed(); }
    public void attack(Monster m)  { type.attack(m); }
}


public static void main(String[] args) {
    final Species RAT = new Species(10, 10, 4);
    final Species GNOLL = new Species(...);

    Monster harald = new Monster(RAT);
    Monster eve = new Monster(GNOLL);
}
```

Statt des Enums nimmt man eine “echte” Klasse mit Methoden für die Type-Objects. Davon legt man zur Laufzeit Objekte an (das sind dann die möglichen Monster-Typen) und bestückt damit die zu erzeugenden Monster.

Im Monster selbst rufen die Monster-Methoden dann einfach nur die Methoden des Type-Objects auf (Delegation => Strategie-Pattern). Man kann aber auch Attribute im Monster selbst pflegen und durch das Type-Object nur passend initialisieren.

Vorteil: Änderungen erfolgen bei der Parametrisierung der Objekte (an **einer** Stelle im Code, vermutlich `main()` oder beispielsweise durch Einlesen einer Konfig-Datei).

4 Fabrikmethode für die Type-Objects

```
public final class Species {
    ...

    public Monster newMonster() {
        return new Monster(this);
```

```
    }
}


public static void main(String[] args) {
    final Species RAT = new Species(10, 10, 4);
    final Species GNOLL = new Species(...);

    Monster harald = RAT.newMonster();
    Monster eve = GNOLL.newMonster();
}
```

Das Hantieren mit den Type-Objects und den Monstern ist nicht so schön. Deshalb kann man in der Klasse für die Type-Objects noch eine Fabrikmethode (=> Factory-Method-Pattern) mit einbauen, über die dann die Monster erzeugt werden.

5 Vererbung unter den Type-Objects

```
public final class Species {
    ...

    public Species(int attackDamage, int movementSpeed, int xp) {
        this.attackDamage = attackDamage;  this.movementSpeed = movementSpeed;
            this.xp = xp;
    }
    public Species(Species parent, int attackDamage) {
        this.attackDamage = attackDamage;
        movementSpeed = parent.movementSpeed;  xp = parent.xp;
    }
}


public static void main(String[] args) {
    final Species RAT = new Species(10, 10, 4);
    final Species BOSS_RAT = new Species(RAT, 100);
    final Species GNOLL = new Species(...);

    Monster harald = RAT.newMonster();
    Monster eve = GNOLL.newMonster();
}
```

Es wäre hilfreich, wenn die Type-Objects Eigenschaften untereinander teilen/weitergeben könnten. Damit man aber jetzt nicht hier eine tiefe Vererbungshierarchie aufbaut und damit wieder am Anfang des Problems wäre, baut man die Vererbung quasi selbst ein über eine Referenz auf ein Eltern-Type-Object. Damit kann man zur Laufzeit einem Type-Object sagen, dass es bestimmte Eigenschaften von einem anderen Type-Object übernehmen soll.

Im Beispiel werden die Eigenschaften `movementSpeed` und `xp` “vererbt” und entsprechend aus dem Eltern-Type-Object übernommen (sofern dieses übergeben wird).

6 Erzeugen der Type-Objects dynamisch über eine Konfiguration

```
{
    "Rat": {
        "attackDamage": 10,
        "movementSpeed": 10,
        "xp": 4
    },
    "BossRat": {
        "parent": "Rat",
        "attackDamage": 100
    },
    "Gnoll": {
        "attackDamage": ...,
        "movementSpeed": ...,
        "xp": ...
    }
}
```

Jetzt kann man die Konfiguration der Type-Objects in einer Konfig-Datei ablegen und einfach an einer passenden Stelle im Programm einlesen. Dort werden dann damit die Type-Objects angelegt und mit Hilfe dieser dann die passend konfigurierten Monster (und deren Unterarten) erzeugt.

7 Vor- und Nachteile des Type-Object-Pattern

7.1 Vorteil

Es gibt nur noch wenige Klassen auf Code-Ebene (im Beispiel: 2), und man kann über die Konfiguration beliebig viele Monster-Typen erzeugen.

7.2 Nachteil

Es werden zunächst nur Daten “überschrieben”, d.h. man kann nur für die einzelnen Typen spezifische Werte mitgeben/definieren.

Bei Vererbung kann man in den Unterklassen nahezu beliebig das Verhalten durch einfaches Überschreiben der Methoden ändern. Das könnte man in diesem Entwurfsmuster erreichen, in dem man beispielsweise eine Reihe von vordefinierten Verhaltensarten implementiert, die dann anhand von Werten ausgewählt und anhand anderer Werte weiter parametrisiert werden.

7.3 Verwandtschaft zum Flyweight-Pattern

Das Type-Object-Pattern ist keines der “klassischen” Design-Pattern der “Gang of Four” (Gamma u. a. 2011). Dennoch ist es gerade in der Spiele-Entwicklung häufig anzutreffen.

Das Type-Object-Pattern ist sehr ähnlich zum Flyweight-Pattern. In beiden Pattern teilen sich mehrere Objekte gemeinsame Daten, die über Referenzen auf gemeinsame Hilfsobjekte eingebunden werden. Die Zielrichtung unterscheidet sich aber deutlich:

- Beim Flyweight-Pattern ist das Ziel vor allem die Erhöhung der Speichereffizienz, und die dort geteilten Daten müssen nicht unbedingt den "Typ" des nutzenden Objekts definieren.
- Beim Type-Objekt-Pattern ist das Ziel die Flexibilität auf Code-Ebene, indem man die Anzahl der Klassen minimiert und die Typen in ein eigenes Objekt-Modell verschiebt. Das Teilen von Speicher ist hier nur ein Nebeneffekt.

8 Wrap-Up

Type-Object-Pattern: Implementierung eines eigenen Objekt-Modells

- Ziel: Minimierung der Anzahl der Klassen
- Ziel: Erhöhung der Flexibilität

- Schiebe "Typen" in ein eigenes Objekt-Modell
- Type-Objects lassen sich dynamisch über eine Konfiguration anlegen
- Objekte erhalten eine Referenz auf "ihr" Type-Object
- "Vererbung" unter den Type-Objects möglich

9 Zum Nachlesen

- Nystrom (2014, Kap. 13)

Lernziele

- k2: Verschieben des Typ-definierenden Teils der Eigenschaften in ein Type-Object
- k2: Erklären der Ähnlichkeit zum Flyweight-Pattern
- k3: Praktischer Einsatz des Type-Object-Patterns

Quizzes

- Quiz Type-Object-Pattern (ILIAS)

Challenges

Betrachten Sie das folgende `IMonster`-Interface:

```
1 public interface IMonster {
2     String getVariety();
3     int getXp();
4     int getMagic();
5     String makeNoise();
6 }
```

Leiten Sie von diesem Interface eine Klasse `Monster` ab. Nutzen Sie das Type-Object-Pattern und erzeugen Sie verschiedene "Klassen" von Monstern, die sich in den Eigenschaften `variety`, `xp` und `magic` unterscheiden und in der Methode `makeNoise()` entsprechend unterschiedlich verhalten. Die Eigenschaft `xp` wird dabei von jedem Monster während seiner Lebensdauer selbst verwaltet, die anderen Eigenschaften bleiben während der Lebensdauer eines Monsters konstant (ebenso wie die Methode `makeNoise()`).

1. Was wird Bestandteil des Type-Objects? Begründen Sie Ihre Antwort.
2. Implementieren Sie das Type-Object und integrieren Sie es in die Klasse `Monster`.
3. Implementieren Sie eine Factory-Methode in der Klasse für die Type-Objects, um ein neues Monster mit diesem Type-Objekt erzeugen zu können.
4. Implementieren Sie einen "Vererbungs"-Mechanismus für die Type-Objects (nicht Vererbung im Java-/OO-Sinn!). Dabei soll eine Eigenschaft überschrieben werden können.
5. Erzeugen Sie einige Monstertypen und jeweils einige Monster und lassen Sie diese ein Geräusch machen (`makeNoise()`).
6. Ersetzen Sie das Type-Object durch ein selbst definiertes (komplexes) Enum.

Quellen

Gamma, E., R. Helm, R. E. Johnson, und J. Vlissides. 2011. *Design Patterns*. Addison-Wesley.

Nystrom, R. 2014. *Game Programming Patterns*. Genever Benning. https://github.com/munificent/game-programming-patterns.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 71232c0 (tooling: shift headings (use h1 as top-level headings), 2025-04-29)